

POWER BI INTERVIEW PLAYBOOK

(Q&A)

(Freshers + Intermediate + Advanced + Scenarios)

✓ SECTION 1 — 50 MOST FREQUENTLY ASKED POWER BI QUESTIONS

1. What is Power BI?

Power BI is Microsoft's analytics platform used to connect to different data sources, clean and transform data, build models, and create interactive dashboards for business insights.

2. Components of Power BI

- Power BI Desktop
 - Power BI Service
 - Power BI Mobile
 - Power Query
 - Power Pivot
 - Data Gateway
 - Power BI Report Server
-

3. Difference between Power BI Desktop and Power BI Service

Desktop is for building and modeling reports. Service is for publishing, sharing, refreshing data, and collaboration.



Desktop	Service
Build reports	Publish, Share, Refresh
Free	Cloud-based
Modeling, DAX	Collaboration, Apps

4. What is Power Query?

Power Query is the ETL layer used to clean, transform, and reshape data before loading into the data model→ Uses **M language**.

5. What is DAX?

A formula language used for **calculations**, **aggregations**, and **analytics** in Power BI.

6. What is a Measure?

A measure is a dynamic calculation evaluated at query time, typically used in visuals.
e.g. SUM(Sales[Amount]).

7. What is a Calculated Column?

A calculated column is computed row-by-row and stored in the model.

8. Difference between Measure and Calculated Column

Measures are dynamic and evaluate based on filters; calculated columns are fixed and stored in memory.

Measure	Column
Dynamic	Static



Saves memory Memory heavy

For visuals For slicers/relationships

9. What is a Relationship in Power BI?

A relationship connects two tables using key columns to enable filtering between them.

10. Cardinality types

- One-to-Many (1:*)
 - Many-to-One
 - Many-to-Many (:)
 - One-to-One
-

11. What is a Star Schema?

A model where a central fact table connects to multiple dimension tables, improving performance and simplicity.

Model with:

- **Fact tables** (sales, amount)
 - **Dimension tables** (product, customer)
-

12. Why star schema is recommended?

It improves performance, reduces complexity, and avoids ambiguous filtering paths.

- Better performance
- Simpler DAX

- Less ambiguity
-

13. What is a Fact Table?

A table that stores numeric, transactional data such as sales or quantity.

14. What is a Dimension Table?

A table that stores descriptive attributes like product names, customers, or dates.

15. What is a Data Gateway?

A gateway connects on-premise data sources to Power BI Service for refresh.

16. Types of gateways

- Personal gateway
 - Standard(Enterprise) gateway (recommended)
-

17. What is Row Level Security (RLS)?

Row Level Security restricts what data a user can see based on defined roles.

18. What is a Dashboard?

A single-page summary of pinned visuals available only in Power BI Service.



19. What is a Report?

A multi-page, interactive visualization created in Power BI Desktop.

20. What is DirectQuery?

A live connection mode where data remains in the source and queries execute in real time.

21. DirectQuery vs Import

Import is fast and stored in memory. DirectQuery loads data on demand but may be slower.

DirectQuery	Import
Real-time	Fast
Limited DAX	Full features
Slow	In-memory

22. What is Composite Model?

A model that combines Import and DirectQuery tables in one dataset.

23. What is a KPI visual?

A KPI visual shows performance toward a target value. (e.g., Sales vs Target).

24. What is Drilldown?

A way to explore data from summary to detail levels.



25. What is Drillthrough?

Navigating to another page to see detailed information for a selected data point.

26. What is Tooltip?

A small pop-up that shows additional information when hovering over a visual..

27. What is ALL() in DAX?

ALL removes filters from a column or table.

28. What is FILTER() in DAX?

FILTER returns a table filtered by conditions.

29. SUM vs SUMX

SUM aggregates a column; SUMX iterates row-by-row before summing.

- SUM → sums one column
 - SUMX → row-by-row calculation before summing
-

30. What is SAMEPERIODLASTYEAR?

Returns the same period from the previous year, used for YoY comparisons.

31. What is CALCULATE()?

CALCULATE modifies filter context and enables advanced calculations.



32. What is Row Context?

A context where expressions evaluate one row at a time. (Operates row-by-row like in Calculated Columns).

33. What is Filter Context?

Filters applied by visuals, slicers, or CALCULATE that affect results.

34. What is Context Transition?

CALCULATE converts row context into filter context.

35. Merge vs Append in Power Query

Merge joins tables horizontally; Append stacks tables vertically.

- Merge → Join tables
 - Append → Stack tables (Union)
-

36. Removing duplicates

Use Power Query → Remove Rows → Remove Duplicates.

37. What is Query Folding?

Power Query pushes transformations back to the source for efficiency.



38. What is Incremental Refresh?

Only new and changed data is refreshed instead of full dataset reload.

Refresh only new data → used for large datasets.

39. What is a Workspace?

A collaborative environment in Power BI Service for reports, dashboards and datasets.

40. What is Shared Dataset?

A published dataset that multiple reports can reuse.

41. Page-Level Filters

Filters applied only to a single page.

42. Visual-Level Filters

Filters applied only to a single visual.

43. Report-Level Filters

Filters that affect the entire report.

44. What are Quick Measures?

Prebuilt DAX formulas created automatically through a menu.



45. What is Bookmark?

A saved state of a report including visuals and filters.

46. What is a Hierarchy?

A structured path such as Country → State → City.

47. What is Data Profiling?

Tools that show column quality, distribution, and missing values.

48. What is Unpivot?

A transformation that turns columns into rows.

49. What is the VertiPaq Engine?

Power BI's in-memory compression engine that speeds up calculations.

50. What file formats does Power BI support?

Excel, CSV, JSON, XML, PDF, SQL Server, SharePoint, etc.



✓ SECTION 2 — ADVANCED POWER BI QUESTIONS

1. What is Evaluation Context?

It's the combined effect of row context and filter context during a DAX calculation.

Evaluation context = **Filter Context + Row Context**.

DAX calculates results by combining both.

2. What is Context Transition? Why does CALCULATE() matter?

CALCULATE converts row context into filter context, enabling dynamic calculations.

Context Transition = converting **row context** → **filter context**, triggered by CALCULATE or CALCULATETABLE.

3. What is a Many-to-Many relationship?

Both sides contain duplicate keys; solved using a bridge table or TREATAS.

[Occurs when both tables have duplicate matching keys.

Common fixes:

- Bridge table
 - Bi-directional filtering
 - CROSSFILTER
 - TREATAS]
-

4. What is Auto Date/Time?

A feature that auto-generates hidden date tables; not recommended for real models.

[Power BI automatically creates hidden date tables for time intelligence.

Not recommended for real projects → use a manual Date table.]

5. What are Role-Playing Dimensions?

One dimension used multiple times, such as Order Date and Ship Date.

[When one dimension is used multiple times in a fact table.

Example: Date table used as:

- Order Date
 - Ship Date
 - Delivery Date]
-

6. What is a Surrogate Key?

A system-generated unique key (integer) used to create relationships between tables.

7. What is a Factless Fact Table?

A fact table with only keys, used for events or tracking without numeric measures.

[Fact table with **only keys, no numeric measures**.

Used for:

- Attendance logging
- Events tracking
- Coverage analysis]



8. What is an Aggregation Table?

A pre-summarized table used to improve performance on large datasets.

[A pre-aggregated summary table used to speed up large datasets.
The engine chooses aggregated table automatically when possible.]

9. What causes relationship ambiguity?

Multiple filter paths between two tables; solved using star schema or inactive relationships.

[Multiple valid paths between tables → Power BI cannot determine filtering direction.
Fix using:

- Star schema
 - Bridge table
 - Inactive relationships]
-

10. How do you optimize Power BI performance?

- Follow **Star Schema**
- Remove high-cardinality columns
- Convert columns → measures where possible
- Disable unnecessary auto-date tables
- Use fewer visuals
- Use DirectQuery only when required
- Optimize DAX (avoid iterators where possible)

- Use aggregation tables
 - Reduce model size by removing unused columns
-

11. What is Bidirectional Cross Filtering?

Filtering flows both ways; used carefully in many-to-many modeling.

[Filters flow **in both directions**.

Helpful for:

- Many-to-many
- Complex hierarchies

Risk:

- Can cause ambiguity and incorrect totals.]
-

12. What is CROSSFILTER in DAX?

Overrides relationship direction inside a calculation.

[Overrides the default relationship direction during measure evaluation.

Example:

```
CALCULATE([Sales], CROSSFILTER(Sales[ProductID], Product[ProductID],  
BOTH))]
```

13. How do you handle Many-to-Many in Power BI?

- Create a bridge table
- Use bi-directional filtering only when safe
- Use CROSSFILTER



- Use TREATAS to simulate relationships
-

14. What is a Composite Model?

A model combining Import and DirectQuery modes.

[A model with multiple storage modes:

- Import
- DirectQuery
- Hybrid (DQ + Import)

Supports large-scale, real-time architectures.]

15. What are Calculation Groups?

They simplify repeated logic such as YTD, YoY, or formatting for all measures.

[Created using Tabular Editor.

Used to avoid repeating measures (YTD, YoY, MoM, Currency Conversion).]

16. What is Auto-exist behavior?

Power BI automatically removes impossible combinations from dimension tables to improve performance.

17. What is a Degenerate Dimension?

A dimension that exists inside a fact table, like invoice numbers.



18. What is Query Folding?

Power Query delegating transformations back to the source for performance.

[Power Query pushes transformations back to the source.

Breaks for:

- Index column
 - Complex M code
 - Non-foldable sources]
-

19. What is a Snowflake Schema? Why is it not recommended?

Dimensions with sub-dimensions; not preferred due to slower performance.

[Dimension tables split into sub-dimensions.

Not recommended due to:

- Slower performance
 - Complex relationships
 - Harder DAX]
-

20. How do you handle slow visuals?

- Reduce number of visuals
- Avoid high-cardinality slicers
- Use summarize tables
- Avoid complex calculations
- Convert complex charts into simpler ones
- Replace maps with simpler visuals



21. What is SELECTEDMEASURE()?

Used in Calculation Groups to apply logic to the active measure.

22. What are Virtual Tables in DAX?

Tables created in DAX using functions like SUMMARIZE or ADDCOLUMNS.

23. What is TREATAS()?

Creates virtual relationships between tables inside DAX.

24. How do you detect circular dependency?

When formulas depend on each other; solved by restructuring formulas or relationships.

[Occurs when calculated columns/measures depend on each other.

Fix by:

- Removing bi-directional filter
 - Refactoring formulas
 - Using bridge tables]
-

25. How do you implement Dynamic RLS?

Use a mapping table and USERPRINCIPALNAME() to filter data per user..

26. Difference between REMOVEFILTERS() and ALL()?

REMOVEFILTERS clears filters; ALL clears filters and returns full table.

REMOVEFILTERS	ALL
---------------	-----

Removes filters	Removes filters + returns full table
-----------------	--------------------------------------

New standard	Legacy approach
--------------	-----------------

27. Explain the use of USERPRINCIPALNAME().

Returns the current user's email; used in security models.

28. How do you handle large datasets (100M+ rows)?

- Aggregation tables
 - Partitioning
 - Incremental refresh
 - DirectQuery + Aggregations
 - Field parameters for performance
 - Optimize model cardinality
 - Composite models
-



29. What is Incremental Refresh?

Refreshes only recent partitions instead of the entire dataset.

[Refreshes only **new** and **changed** data instead of full reload.]

30. Explain difference between SUMX and CALCULATE(SUM()).

SUMX iterates row-by-row; CALCULATE adjusts filter context.

SUMX → iterator (row-by-row)

CALCULATE(SUM()) → modifies filter context

✓ SECTION 3 — SCENARIO-BASED QUESTIONS (REAL-TIME)

Scenario 1: Table visual shows wrong total

Issue: Using SUM instead of SUMX

Fix:

`Total Sales = SUMX(Sales, Sales[Qty] * Sales[Price])`

Use SUMX to properly evaluate row-level calculations.

Scenario 2: Slicer not filtering report

Fix:

- Check relationships
- Check cross-filter direction
- Make sure both tables are connected via a dimension



Scenario 3: Report is slow. What will you do?

Solutions:

- Remove unnecessary columns
 - Reduce model size
 - Use Import instead of DirectQuery
 - Use aggregated tables
 - Use **star schema**
 - Reduce visuals
 - Simplify DAX
 - Avoid row-level calculated columns
 - Replace **Columns** with **Measures**
-

Scenario 4: You need YoY growth. How will you calculate it?

Answer:

Use SAMEPERIODLASTYEAR with CALCULATE.

- Create a **Date Table** (marked as Date Table)
- Use `TOTALYTD()` or `SAMEPERIODLASTYEAR()`
- Example:

```
YoY Sales =  
SUM(Sales[Amount]) - CALCULATE(SUM(Sales[Amount]),  
SAMEPERIODLASTYEAR('Date'[Date]))
```



Scenario 5: Need to filter data by user

Use **RLS** roles(Implement RLS or dynamic RLS).

Scenario 6: Need dynamic title

Use SELECTEDVALUE in a measure.

Title = "Sales for " & SELECTEDVALUE('Date'[Month])

Scenario 7: Duplicate Customer IDs

Use:

Remove duplicates in Power Query

Scenario 8: Merge multiple Excel sheets

Use Append and Combine Files.

Scenario 9: Fact table has 2 date columns

Use role-playing dimensions(Create **two roles** from the Date dimension).

Scenario 10: Data refresh failing

Check:

- Gateway connectivity
- Credentials



- Data source permissions
-

Scenario 11: Your data has duplicate rows. How will you remove them?

Answer:

- Open **Power Query** → Home → Remove Rows → Remove Duplicates
 - Duplicate removal is done **before loading** the data
-

Scenario 12: Sales table has ProductID but the Product Name is not appearing in visuals. What will you check?

Answer:

- Check if relationship exists between **Sales[ProductID]** and **Product[ProductID]**
 - Ensure cardinality is **Many-to-One**
 - Ensure cross-filter direction is **Single**
-

Scenario 13: A measure is giving wrong results in a table visual but correct in a card. Why?

Answer:

Wrong results in table usually due to:

- Incorrect **filter context**
 - Missing or wrong **relationship**
 - DAX formula not using **CALCULATE()**
-



Scenario 14: You want to limit a user to view only South Region data. How?

Answer:

- Create RLS role: `[Region] = "South"`
 - Assign user to that role in Power BI Service.
-

Scenario 15: You want to refresh data automatically every day.

Answer:

- Publish to Service
 - Set **Scheduled Refresh**
 - If on-prem → Configure **Gateway**
-

✓ SECTION 4 — DAX CHEAT SHEET

Most Used Aggregation Functions

- SUM
- AVERAGE
- COUNT
- DISTINCTCOUNT
- COUNTROWS



Iterators (X Functions)

- SUMX
 - AVERAGEX
 - COUNTX
 - FILTERX
-

Time Intelligence

- SAMEPERIODLASTYEAR
 - TOTALYTD
 - DATESYTD
 - DATEADD
 - STARTOFMONTH
-

Filter Functions

- FILTER
 - ALL
 - ALLEXCEPT
 - ALLSELECTED
 - VALUES
-



Logical Functions

- IF
- SWITCH
- AND / OR / NOT

1 BASIC AGGREGATION FUNCTIONS

A. SUM()

Description:

Adds all values in a numeric column.

Syntax:

```
SUM(Table[Column])
```

Example:

```
Total Sales = SUM(Sales[Amount])
```

Use when:

You need to add all numeric values from a single column.

B. AVERAGE()

Description:

Returns the average of a numeric column.

Syntax:

```
AVERAGE(Table[Column])
```

Example:

```
Avg Sales = AVERAGE(Sales[Amount])
```



C. COUNT()

Description:

Counts the number of non-blank rows in a column.

Syntax:

```
COUNT(Table[Column])
```

Example:

```
Count Products = COUNT(Product[ProductID])
```

D. DISTINCTCOUNT()

Description:

Counts the number of unique values.

Syntax:

```
DISTINCTCOUNT(Table[Column])
```

Example:

```
Unique Customers = DISTINCTCOUNT(Sales[CustomerID])
```

E. COUNTROWS()

Description:

Counts the number of rows in a table.



Syntax:

`COUNTROWS(Table)`

Example:

`Total Orders = COUNTROWS(Sales)`

2 ITERATOR FUNCTIONS (X Functions)

Iterators evaluate expressions **row-by-row**, unlike basic aggregators.

A. SUMX()

Description:

Evaluates each row → returns a result → sums all results.

Syntax:

`SUMX(Table, Expression)`

Example:

`Total Revenue = SUMX(Sales, Sales[Qty] * Sales[Price])`

Use when:

Totals are wrong when using SUM.

B. AVERAGEX()

Description:

Returns average after evaluating an expression row-by-row.



Syntax:

```
AVERAGEX(Table, Expression)
```

Example:

```
Avg Revenue =
```

```
AVERAGEX(Sales, Sales[Qty] * Sales[Price])
```

C. COUNTX()**Description:**

Counts rows where an expression returns a non-blank result.

Syntax:

```
COUNTX(Table, Expression)
```

Example:

Count products with Sales > 1000:

```
High Sales Products =
```

```
COUNTX(Sales, IF(Sales[Amount] > 1000, 1))
```

D. FILTERX() (Not a real function — interviewer expects "using FILTER inside X functions")**Concept Explanation:**

We use **FILTER** inside **SUMX** / **AVERAGEX** / **COUNTX** for filtered iteration.



Example:

High Value Sales =

```
SUMX(  
    FILTER(Sales, Sales[Amount] > 5000),  
    Sales[Amount]  
)
```

3 TIME INTELLIGENCE FUNCTIONS

⚠ Requires a **proper Date table** marked as a Date Table.

A. SAMEPERIODLASTYEAR()

Description:

Returns the same period from the previous year.

Syntax:

```
SAMEPERIODLASTYEAR(Date[Date])
```

Example:

Sales LY =

```
CALCULATE(SUM(Sales[Amount]),  
SAMEPERIODLASTYEAR('Date'[Date]))
```



B. TOTALYTD()

Description:

Returns year-to-date values.

Syntax:

```
TOTALYTD(Expression, DateColumn)
```

Example:

```
YTD Sales = TOTALYTD(SUM(Sales[Amount]), 'Date'[Date])
```

C. DATESYTD()

Description:

Returns the set of dates from the start of the year to the current date.

Syntax:

```
DATESYTD(Date[Date])
```

Example:

```
Sales YTD =  
CALCULATE(SUM(Sales[Amount]),  
DATESYTD('Date'[Date]))
```

D. DATEADD()

Description:

Shifts dates forward or backward.



Syntax:

```
DATEADD(DateColumn, Number, Interval)
```

Example:

Sales last month:

```
Sales LM =
```

```
CALCULATE([Total Sales], DATEADD('Date'[Date], -1, MONTH))
```

E. STARTOFMONTH()

Description:

Returns the first date of the month.

Syntax:

```
STARTOFMONTH(Date[Date])
```

Example:

Sales from start of month:

```
Sales From Start =
```

```
CALCULATE([Total Sales],  
    STARTOFMONTH('Date'[Date])  
)
```



4 FILTER FUNCTIONS

A. FILTER()

Description:

Creates a filtered table based on condition.

Syntax:

```
FILTER(Table, Condition)
```

Example:

High Sales =

```
CALCULATE([Total Sales], FILTER(Sales, Sales[Amount] > 10000))
```

B. ALL()

Description:

Removes filters from a table or column.

Syntax:

```
ALL(Table/Column)
```

Example:

% of Total:

% Sales =

```
DIVIDE(SUM(Sales[Amount]),  
        CALCULATE(SUM(Sales[Amount]), ALL(Sales)))
```

C. ALLEXCEPT()

Description:

Removes all filters **except** specified columns.

Syntax:

```
ALLEXCEPT(Table, Column)
```

Example:

Keep category, remove all other filters:

```
Category Sales =  
CALCULATE(SUM(Sales[Amount]),  
           ALLEXCEPT(Product, Product[Category]))
```

D. ALLSELECTED()

Description:

Keeps external filters but ignores internal filters.

Syntax:

```
ALLSELECTED(Table/Column)
```

Example:

Use in dynamic ranking:

```
Rank =  
RANKX(ALLSELECTED(Product[ProductName]), [Total Sales])
```



E. VALUES()

Description:

Returns unique values from a column.

Syntax:

```
VALUES(Table[Column])
```

Example:

```
Selected Product = VALUES(Product[ProductName])
```

5 LOGICAL FUNCTIONS

A. IF()

Description:

Returns result based on logical condition.

Syntax:

```
IF(Condition, Result1, Result2)
```

Example:

```
Status = IF([Sales] > 10000, "High", "Low")
```

B. SWITCH()

Description:

Replaces multiple IF statements.



Syntax:

```
SWITCH(Expression, Value1, Result1, Value2, Result2)
```

Example:

```
Rating =
```

```
SWITCH(TRUE(),  
    [Sales] > 100000, "A",  
    [Sales] > 50000, "B",  
    "C")
```

C. AND / OR / NOT**Syntax:**

```
AND(condition1, condition2)
```

```
OR(condition1, condition2)
```

```
NOT(condition)
```

Example:

```
High-value sales:
```

```
High Value =
```

```
IF( AND(Sales[Amount] > 10000, Sales[Qty] > 5), "High", "Normal")
```



✓ SECTION 5 — MODELING CHEAT SHEET

Best Practices

- Use **Star Schema**
- Avoid snowflake when possible
- Remove unnecessary columns
- Reduce cardinality
- Create Date dimension manually
- Only 1 active relationship per table
- Use surrogate keys
- Prefer measures over columns

1. Use Star Schema (MOST IMPORTANT FOR INTERVIEWS)

What is Star Schema?

A data model where **one central Fact table** (transactions) connects to multiple **Dimension tables** (attributes).

Why Star Schema?

- Best performance (VertiPaq compression works better)
- Cleaner, simpler relationships
- Easier DAX
- Avoids ambiguity
- Less memory usage
- Recommended by Microsoft



Example:

FactSales → connected to:

- DimDate
- DimProduct
- DimCustomer
- DimRegion

In simple words:

Always follow a star schema because it gives faster performance, cleaner relationships, and avoids ambiguity in DAX calculations.

2. Avoid Snowflake Schema When Possible

What is Snowflake Schema?

A dimension table split into sub-dimensions.

Example (Bad Model):

Product → SubCategory → Category → Brand

Why avoid it?

- More relationships → slower performance
- Complex DAX
- Harder to understand
- Higher chance of relationship ambiguity

When is Snowflake okay?

Only when:

- Data is extremely normalized



- You cannot change the source
- Table size is very large

In simple words:

Avoid snowflake schema because it adds unnecessary joins and slows the model. Power BI works best with flattened dimension tables.

3. Remove Unnecessary Columns

Why remove them?

- Reduces memory usage
- Improves refresh time
- Improves model load speed
- Reduces clutter
- Enhances performance

What to remove?

- Columns never used in visuals
- Audit columns (CreatedBy, UpdatedOn, etc.)
- High-cardinality text (unless required)
- Duplicate descriptive fields

In simple words:

Remove unused columns to improve model performance and reduce memory consumption. Only needed columns stay in the model.



4. Reduce Cardinality

What is cardinality?

The number of unique values in a column.

High Cardinality Examples:

- Customer Email
- GUIDs
- Transaction IDs
- DateTime with seconds

Why reduce it?

High cardinality:

- Slows down the model
- Takes more memory
- Reduces compression efficiency
- Slows visuals & filters

How to reduce it?

- Split DateTime → Date + Time
- Convert long text → Code (e.g., Category Code)
- Avoid using unique IDs in slicers
- Remove unnecessary unique identifiers

In simple words:

Always reduce cardinality because high-cardinality columns hurt performance and compression.



5. Create Date Dimension Manually

Why not Auto Date/Time?

Auto Date/Time creates hidden date tables:

- Not optimized
- Consumes memory
- Difficult for custom time intelligence
- Bad for enterprise models

A Manual Date Table Allows:

- Fiscal calendar
- Custom holidays
- MTD, YTD, QTD
- Time intelligence consistency

Example Date Table Columns:

- Date
- Year, Month, Quarter
- Week Number
- Fiscal Year, Fiscal Quarter

In simple words:

Always create a manual Date dimension and mark it as a Date Table to use advanced time intelligence properly.



6. Only ONE Active Relationship per Table

Why only one active relationship?

Active relationships affect filter propagation.

If more than one exists → Ambiguity

Power BI rule:

Only **one active** relationship allowed between two tables.

Example Scenario:

FactSales has:

- OrderDate
- ShipDate

You create two relationships:

- OrderDate → Active
- ShipDate → Inactive

Switch using USERELATIONSHIP():

```
Sales by Ship Date =  
CALCULATE([Total Sales],  
USERELATIONSHIP(Sales[ShipDate], Date[Date]))
```

In simple words:

Maintain only one active relationship and use inactive relationships with USERELATIONSHIP() when needed.

7. Use Surrogate Keys

What are Surrogate Keys?

System-generated integer keys (like ID fields).



Why use them?

- Faster relationships
- Better compression
- Avoid data type mismatches
- Handle slowly changing dimensions (SCD)

Example:

ProductKey (Surrogate Key)

Instead of using long ProductCode or text fields.

In simple words:

Prefer surrogate integer keys because they improve performance and avoid issues with natural keys.

8. Prefer Measures Over Calculated Columns

Why prefer measures?

- Measures are dynamic
- Computed at query time
- Do not take memory
- Adjust based on filters
- Faster compression

Why avoid too many calculated columns?

- Stored in memory
- Increase model size
- Slow refresh



- Not filter-responsive

In simple words:

Always use measures instead of calculated columns unless row-level computation is required.

SUMMARY

- Modeling approach is always performance-first.
- Follow star schema and avoid snowflake for simplicity and speed.
- Remove unused columns, reduce cardinality, and use surrogate keys to improve compression.
- Create a manual date table for proper time intelligence.
- Ensure only one active relationship is used and prefer measures over calculated columns to keep the model lightweight and fast.

POWER BI – LAST MINUTE INTERVIEW REVISION SHEET

1. POWER BI CORE BASICS

- Power BI = Data visualization + modeling + analytics
 - Components = Desktop, Service, Gateway, Mobile, Power Query, DAX
 - Report = Multi-page, built in Desktop
 - Dashboard = Single page, pinned visuals (Service only)
-

2. POWER QUERY (ETL)

- Clean/transform data before loading
 - Merge = Join tables
 - Append = Union tables
 - Remove duplicates = Power Query → Home → Remove Duplicates
 - Unpivot = Convert columns → rows
 - Query folding = Pushes transformations to source (important for performance)
-

3. DATA MODELING QUICK NOTES

- Always use **Star Schema**
- Fact table = numeric measures (sales)
- Dimension table = descriptive attributes (product, customer)



- Relationship: Fact → Dimension (Many-to-One)
 - Only **one active** relationship
 - Prefer **surrogate keys** (integer join keys)
 - Role-playing dimensions = Same dimension used multiple times (Order Date, Ship Date)
-

4. STORAGE MODES

- Import = Fast, in-memory (recommended)
 - DirectQuery = Live, slower
 - Composite model = Import + DirectQuery
 - Incremental refresh = Only new data loads
-

5. DAX IN 60 SECONDS

Must-know functions:

- SUM, AVERAGE, COUNTROWS
- CALCULATE → changes filter context
- FILTER → returns filtered table
- SUMX → row-by-row calculation
- SAMEPERIODLASTYEAR → YoY
- ALL / REMOVEFILTERS → remove filters
- USERPRINCIPALNAME → RLS



Contexts:

- Row context = Row-by-row
 - Filter context = Applied by visuals
 - Context transition = CALCULATE()
-

6. MOST ASKED DAX PATTERNS

YoY:

```
YoY Sales = SUM(Sales[Amount]) - CALCULATE(SUM(Sales[Amount]),  
SAMEPERIODLASTYEAR('Date'[Date]))
```

Correct total (SUMX):

```
Total Sales = SUMX(Sales, Sales[Qty] * Sales[Price])
```

Dynamic Title:

```
Title = "Sales for " & SELECTEDVALUE('Date'[Month])
```

7. PERFORMANCE TUNING QUICK RULES

- Use **star schema**
- Avoid high-cardinality columns
- Replace columns with **measures**
- Remove unused columns
- Reduce visuals



- Use Import mode if possible
 - Use **aggregation tables** for large data
 - Real-time = DirectQuery + Aggregations
 - Query folding should remain ON
-

8. RLS (ROW LEVEL SECURITY)

Static:

```
[Region] = "South"
```

Dynamic:

Use mapping table +

```
USERPRINCIPALNAME()
```

9. POWER BI SERVICE

- Publish report → Create workspace
 - Scheduled refresh = Daily/Hourly
 - Gateway needed for on-premises data
 - Apps = For sharing to end-users
-

10. SCENARIOS (SUPER IMPORTANT)

Memorize these — they are asked in **every interview**:

- ✓ **Slicer not filtering** → Fix relationship / cross-filter direction
- ✓ **Wrong total** → Use SUMX
- ✓ **Duplicate rows** → Remove in Power Query



- ✓ **Sales not showing product name** → Relationship missing
 - ✓ **Report slow** → Reduce cardinality, remove columns, star schema
 - ✓ **Two date columns** → Use role-playing dimensions
 - ✓ **Refresh failing** → Gateway / credentials / data source
 - ✓ **Show only user's region** → RLS
-

11. ADVANCED MUST-KNOW

- TREATAS → Virtual relationship
- CROSSFILTER → Override relationship direction
- Auto-exist → Removes impossible combinations
- Calculation groups → YTD/MoM/YoY for all measures
- Virtual tables → SUMMARIZE, ADDCOLUMNS

